

-- =====

-- MySQL Storage Engines Hands-on Lab

-- Engines: InnoDB, MyISAM, MEMORY, CSV, ARCHIVE

-- =====

DROP DATABASE IF EXISTS storage_engine_lab;

CREATE DATABASE storage_engine_lab;

USE storage_engine_lab;

-- Check supported engines

11 • SHOW ENGINES;

Result Grid  Filter Rows: Export:  Wrap Cell Content: 						
	Engine	Support	Comment	Transactions	XA	Savepoints
▶	MEMORY	YES	Hash based, stored in memory, usefu...	NO	NO	NO
	MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
	CSV	YES	CSV storage engine	NO	NO	NO
	FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL
	PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
	MyISAM	YES	MyISAM storage engine	NO	NO	NO
	InnoDB	DEFAULT	Supports transactions, row-level locki...	YES	YES	YES
	ndbinfo	NO	MySQL Cluster system information st...	NULL	NULL	NULL
	BLACKHOLE	YES	/dev/null storage engine (anything y...	NO	NO	NO
	ARCHIVE	YES	Archive storage engine	NO	NO	NO
	ndbcluster	NO	Clustered, fault-tolerant tables	NULL	NULL	NULL

-- Support means Indicates whether the storage engine is available and usable in MySQL.

-- XA means Indicates whether the engine supports XA Transactions (Distributed Transactions).

-- Savepoints means Indicates whether the engine supports SAVEPOINT inside transactions.

```
-- =====
```

```
-- 1. CREATE TABLES WITH DIFFERENT STORAGE ENGINES
```

```
-- =====
```

```
CREATE TABLE customers_innodb (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    email VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE customers_myisam (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    email VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=MyISAM;
```

```
CREATE TABLE customers_memory (  
    customer_id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    email VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=MEMORY;
```

```
CREATE TABLE customers_csv (
    customer_id INT NOT NULL,
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=CSV;
```

```
CREATE TABLE customers_archive (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100),
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=ARCHIVE;
```

-- Check table engines

56 • **SHOW TABLE STATUS WHERE Name LIKE 'customers_%';**

Result Grid Filter Rows: Export: Wrap Cell Content: IA									
	Name	Engine	Version	Row_format	Rows	Avg_row_length	Data_length	Max_data_length	Index_le
▶	customers_archive	ARCHIVE	10	Compressed	0	813	88	0	0
	customers_csv	CSV	10	Dynamic	2	0	0	0	0
	customers_innodb	InnoDB	10	Dynamic	0	0	16384	0	0
	customers_memory	MEMORY	10	Fixed	0	813	0	16237236	0
	customers_myisam	MyISAM	10	Dynamic	0	0	0	281474976710655	1024

```
-- =====
```

```
-- 2. INSERT SAMPLE DATA
```

```
-- =====
```

```
INSERT INTO customers_innodb (customer_name, email)
```

```
VALUES
```

```
('Aung Aung', 'aung@example.com'),
```

```
('Su Su', 'susu@example.com'),
```

```
('Kyaw Kyaw', 'kyaw@example.com');
```

```
INSERT INTO customers_myisam (customer_name, email)
```

```
SELECT customer_name, email FROM customers_innodb;
```

```
INSERT INTO customers_memory (customer_name, email)
```

```
SELECT customer_name, email FROM customers_innodb;
```

```
INSERT INTO customers_csv (customer_id, customer_name, email)
```

```
VALUES
```

```
(1, 'Aung Aung', 'aung@example.com'),
```

```
(2, 'Su Su', 'susu@example.com'),
```

```
(3, 'Kyaw Kyaw', 'kyaw@example.com');
```

```
INSERT INTO customers_archive (customer_name, email)
```

```
SELECT customer_name, email FROM customers_innodb;
```

-- View data

84 • **SELECT * FROM** customers_innodb;

Result Grid				
Filter Rows:				
Edit:   				
Export/Import:  				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:13
	2	Su Su	susu@example.com	2026-05-11 14:49:13
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:13
*	NULL	NULL	NULL	NULL

85 • **SELECT * FROM** customers_myisam;

Result Grid				
Filter Rows:				
Edit:   				
Export/Import:  				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:20
	2	Su Su	susu@example.com	2026-05-11 14:49:20
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:20
*	NULL	NULL	NULL	NULL

86 • **SELECT * FROM** customers_memory;

Result Grid				
Filter Rows:				
Edit:   				
Export/Import:  				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:22
	2	Su Su	susu@example.com	2026-05-11 14:49:22
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:22
*	NULL	NULL	NULL	NULL

87 • **SELECT * FROM** customers_csv;

Result Grid				
Filter Rows:				
Export:  Wrap Cell Content: 				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:25
	2	Su Su	susu@example.com	2026-05-11 14:49:25
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:25

88 • **SELECT * FROM** customers_archive;

Result Grid				
Filter Rows:				
Edit:   				
Export/Import:  				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:29
	2	Su Su	susu@example.com	2026-05-11 14:49:29
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:29
*	NULL	NULL	NULL	NULL

-- =====

-- TABLE SIZE REPORT BY STORAGE ENGINE

-- =====

```
95 • SELECT
96     table_name,
97     engine,
98     table_rows,
99     ROUND(data_length / 1024, 2) AS data_kb,
100    ROUND(index_length / 1024, 2) AS index_kb,
101    ROUND((data_length + index_length) / 1024, 2) AS total_kb,
102    ROUND(data_free / 1024, 2) AS free_kb
103 FROM information_schema.tables
104 WHERE table_schema = 'storage_engine_lab'
105 ORDER BY total_kb DESC;
```

TABLE_NAME	ENGINE	TABLE_ROWS	data_kb	index_kb	total_kb	free_kb
customers_innodb	InnoDB	0	16.00	0.00	16.00	0.00
customers_myisam	MyISAM	0	0.00	1.00	1.00	0.00
customers_archive	ARCHIVE	0	0.09	0.00	0.09	0.00
customers_csv	CSV	2	0.00	0.00	0.00	0.00
customers_memory	MEMORY	0	0.00	0.00	0.00	0.00

-- =====

-- 3. TRANSACTION TEST

-- InnoDB supports ROLLBACK

-- MyISAM, MEMORY, CSV, ARCHIVE do not support real rollback

-- =====

START TRANSACTION;

INSERT INTO customers_innodb (customer_name, email)

VALUES ('Rollback Test InnoDB', 'rollback_innodb@example.com');

118 • **SELECT * FROM** customers_innodb;

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:13
	2	Su Su	susu@example.com	2026-05-11 14:49:13
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:13
	4	Rollback Test InnoDB	rollback_innodb@example.com	2026-05-11 14:54:10
*	NULL	NULL	NULL	NULL

ROLLBACK;

122 • **SELECT * FROM** customers_innodb

123 **WHERE** customer_name = 'Rollback Test InnoDB';

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:				
	customer_id	customer_name	email	created_at
*	NULL	NULL	NULL	NULL

-- MyISAM rollback test

START TRANSACTION;

INSERT INTO customers_myisam (customer_name, email)

VALUES ('Rollback Test MyISAM', 'rollback_myisam@example.com');

131 • **SELECT * FROM** customers_myisam;

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:20
	2	Su Su	susu@example.com	2026-05-11 14:49:20
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:20
	4	Rollback Test MyISAM	rollback_myisam@example.com	2026-05-11 14:55:10
*	NULL	NULL	NULL	NULL

ROLLBACK;

```

134 • SELECT * FROM customers_myisam
135 WHERE customer_name = 'Rollback Test MyISAM';

```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:				
	customer_id	customer_name	email	created_at
▶	4	Rollback Test MyISAM	rollback_myisam@example.com	2026-05-11 14:55:10
*	NULL	NULL	NULL	NULL

-- Expected:

-- InnoDB record disappears after rollback.

-- MyISAM record remains because MyISAM is non-transactional.

-- =====

-- 4. UPDATE TEST

-- =====

```

UPDATE customers_innodb
SET email = 'updated_innodb@example.com'
WHERE customer_id = 1;

```

```

UPDATE customers_myisam
SET email = 'updated_myisam@example.com'
WHERE customer_id = 1;

```

```

UPDATE customers_memory
SET email = 'updated_memory@example.com'
WHERE customer_id = 1;

```

-- CSV may not support UPDATE properly depending on MySQL version

-- Try this:

```
UPDATE customers_csv
```

```
SET email = 'updated_csv@example.com'
```

```
WHERE customer_id = 1;
```

-- ARCHIVE usually does not support UPDATE

```
UPDATE customers_archive
```

```
SET email = 'updated_archive@example.com'
```

```
WHERE customer_id = 1;
```

-- View data

169 • **SELECT * FROM customers_innodb;**

customer_id	customer_name	email	created_at
1	Aung Aung	updated_innodb@example.com	2026-05-11 14:49:13
2	Su Su	susu@example.com	2026-05-11 14:49:13
3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:13
* NULL	NULL	NULL	NULL

170 • **SELECT * FROM customers_myisam;**

customer_id	customer_name	email	created_at
1	Aung Aung	updated_myisam@example.com	2026-05-11 14:49:20
2	Su Su	susu@example.com	2026-05-11 14:49:20
3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:20
4	Rollback Test MyISAM	rollback_myisam@example.com	2026-05-11 14:55:10
* NULL	NULL	NULL	NULL

171 • **SELECT * FROM customers_memory;**

customer_id	customer_name	email	created_at
1	Aung Aung	updated_memory@example.com	2026-05-11 14:49:22
2	Su Su	susu@example.com	2026-05-11 14:49:22
3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:22
* NULL	NULL	NULL	NULL

172 • **SELECT * FROM** customers_csv;

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:25
	2	Su Su	susu@example.com	2026-05-11 14:49:25
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:25

173 • **SELECT * FROM** customers_archive;

Result Grid				
Filter Rows:				
Edit:				
Export/Import:				
	customer_id	customer_name	email	created_at
▶	1	Aung Aung	aung@example.com	2026-05-11 14:49:29
	2	Su Su	susu@example.com	2026-05-11 14:49:29
	3	Kyaw Kyaw	kyaw@example.com	2026-05-11 14:49:29
*	NULL	NULL	NULL	NULL

=====

-- 5. DELETE TEST

-- =====

```
178 • DELETE FROM customers_innodb WHERE customer_id = 2;
179 • DELETE FROM customers_myisam WHERE customer_id = 2;
180 • DELETE FROM customers_memory WHERE customer_id = 2;
181
182 -- CSV / ARCHIVE may give errors depending on version
183 • DELETE FROM customers_csv WHERE customer_id = 2;
184 • DELETE FROM customers_archive WHERE customer_id = 2;
```

Output		
Action Output		
#	Time	Action
✓ 227	14:59:56	DELETE FROM customers_innodb WHERE customer_id = 2
✓ 228	14:59:58	DELETE FROM customers_myisam WHERE customer_id = 2
✓ 229	14:59:59	DELETE FROM customers_memory WHERE customer_id = 2
✗ 230	15:00:02	DELETE FROM customers_csv WHERE customer_id = 2
✗ 231	15:00:04	DELETE FROM customers_archive WHERE customer_id = 2

-- =====

-- 6. PERFORMANCE TEST TABLES

-- =====

```
CREATE TABLE perf_innodb (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    amount DECIMAL(10,2),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    INDEX idx_amount (amount)  
) ENGINE=InnoDB;
```

```
CREATE TABLE perf_myisam (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    amount DECIMAL(10,2),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    INDEX idx_amount (amount)  
) ENGINE=MyISAM;
```

```
CREATE TABLE perf_memory (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    amount DECIMAL(10,2),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    INDEX idx_amount (amount)
```

```
) ENGINE=MEMORY;
```

```
CREATE TABLE perf_archive (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    amount DECIMAL(10,2),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=ARCHIVE;
```

```
-- =====  
-- 7. INSERT PERFORMANCE TEST  
-- Run each block separately and check execution time  
-- in MySQL Workbench Action Output  
-- =====
```

```
INSERT INTO perf_innodb (customer_name, amount)  
SELECT CONCAT('Customer ', n), RAND() * 100000  
FROM (  
    SELECT a.N + b.N * 10 + c.N * 100 + d.N * 1000 AS n  
    FROM  
        (SELECT 0 N UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL  
        SELECT 4  
         UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION  
        ALL SELECT 9) a,  
        (SELECT 0 N UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL  
        SELECT 4
```

UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) b,

(SELECT 0 N UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4

UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) c,

(SELECT 0 N UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4

UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) d

) numbers;

InnoDB - 0.125 sec

INSERT INTO perf_myisam (customer_name, amount)

SELECT customer_name, amount FROM perf_innodb;

MyISAM - 0.344 sec

INSERT INTO perf_memory (customer_name, amount)

SELECT customer_name, amount FROM perf_innodb;

MEMORY - 0.093 sec

INSERT INTO perf_archive (customer_name, amount)

SELECT customer_name, amount FROM perf_innodb;

ARCHIVE - 0.156 sec

-- =====

-- TABLE SIZE REPORT BY STORAGE ENGINE

-- =====

```
255 • SELECT
256     table_name,
257     engine,
258     table_rows,
259     ROUND(data_length / 1024, 2) AS data_kb,
260     ROUND(index_length / 1024, 2) AS index_kb,
261     ROUND((data_length + index_length) / 1024, 2) AS total_kb,
262     ROUND(data_free / 1024, 2) AS free_kb
263 FROM information_schema.tables
264 WHERE table_schema = 'storage_engine_lab'
265 ORDER BY total_kb DESC;
266
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	TABLE_NAME	ENGINE	TABLE_ROWS	data_kb	index_kb	total_kb	free_kb
▶	perf_memory	MEMORY	10000	4256.44	503.78	4760.22	0.00
	perf_innodb	InnoDB	10196	528.00	272.00	800.00	0.00
	perf_myisam	MyISAM	10000	312.50	225.00	537.50	0.00
	perf_archive	ARCHIVE	10000	83.95	0.00	83.95	0.00
	customers_innodb	InnoDB	0	16.00	0.00	16.00	0.00
	customers_myisam	MyISAM	0	0.00	1.00	1.00	0.00
	customers_archive	ARCHIVE	0	0.09	0.00	0.09	0.00
	customers_csv	CSV	2	0.00	0.00	0.00	0.00
	customers_memory	MEMORY	0	0.00	0.00	0.00	0.00

-- Count records

```
268 • SELECT 'InnoDB' AS engine_name, COUNT(*) AS total_rows FROM perf_innodb
269 UNION ALL
270 SELECT 'MyISAM', COUNT(*) FROM perf_myisam
271 UNION ALL
272 SELECT 'MEMORY', COUNT(*) FROM perf_memory
273 UNION ALL
274 SELECT 'ARCHIVE', COUNT(*) FROM perf_archive;
275
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	engine_name	total_rows			
▶	InnoDB	10000			
	MyISAM	10000			
	MEMORY	10000			
	ARCHIVE	10000			

-- =====

-- 8. SELECT PERFORMANCE TEST

-- Run each SELECT separately and compare execution time

-- =====

SELECT * FROM perf_innodb WHERE amount BETWEEN 10000 AND 20000;

InnoDB - 0.000 sec / 0.000 sec

SELECT * FROM perf_myisam WHERE amount BETWEEN 10000 AND 20000;

MyISAM - 0.000 sec / 0.016 sec

SELECT * FROM perf_memory WHERE amount BETWEEN 10000 AND 20000;

MEMORY - 0.015 sec / 0.000 sec

SELECT * FROM perf_archive WHERE amount BETWEEN 10000 AND 20000;

ARCHIVE - 0.000 sec / 0.000 sec

-- Enable Profiling (for MySQL Workbench / Session)

SET profiling = 1;

-- InnoDB

```
293 • SELECT * FROM perf_innodb
294 WHERE amount BETWEEN 10000 AND 50000;
```

Result Grid	Filter Rows:	Edit:	Export/Import:
id	customer_name	amount	created_at
3326	Customer 3624	10014.11	2026-05-11 15:01:40
1379	Customer 1671	10014.30	2026-05-11 15:01:40
431	Customer 539	10016.31	2026-05-11 15:01:40
8966	Customer 8064	10017.89	2026-05-11 15:01:40
5019	Customer 5911	10020.15	2026-05-11 15:01:40
9773	Customer 9773	10031.93	2026-05-11 15:01:40

-- MyISAM

```
297 • SELECT * FROM perf_myisam
298 WHERE amount BETWEEN 10000 AND 50000;
```

Result Grid	Filter Rows:	Edit:	Export/Import:
id	customer_name	amount	created_at
3326	Customer 3624	10014.11	2026-05-11 15:02:02
1379	Customer 1671	10014.30	2026-05-11 15:02:02
431	Customer 539	10016.31	2026-05-11 15:02:02
8966	Customer 8064	10017.89	2026-05-11 15:02:02
5019	Customer 5911	10020.15	2026-05-11 15:02:02
9773	Customer 9773	10031.93	2026-05-11 15:02:02

-- MEMORY

```
301 • SELECT * FROM perf_memory
302 WHERE amount BETWEEN 10000 AND 50000;
```

Result Grid	Filter Rows:	Edit:	Export/Import:
id	customer_name	amount	created_at
4	Customer 906	22418.37	2026-05-11 15:02:06
5	Customer 905	14002.28	2026-05-11 15:02:06
9	Customer 901	37695.98	2026-05-11 15:02:06
10	Customer 900	36668.12	2026-05-11 15:02:06
12	Customer 918	41259.04	2026-05-11 15:02:06
17	Customer 913	15873.01	2026-05-11 15:02:06

-- ARCHIVE

```
305 • SELECT * FROM perf_archive
306 WHERE amount BETWEEN 10000 AND 50000;
```

Result Grid				
Filter Rows:				
Edit:				
Export/Import				
	id	customer_name	amount	created_at
▶	4	Customer 906	22418.37	2026-05-11 15:02:08
	5	Customer 905	14002.28	2026-05-11 15:02:08
	9	Customer 901	37695.98	2026-05-11 15:02:08
	10	Customer 900	36668.12	2026-05-11 15:02:08
	12	Customer 918	41259.04	2026-05-11 15:02:08

-- Get Execution Time (Milliseconds)

-- Show all query profiles

```
310 • SHOW PROFILES;
```

Result Grid			
Filter Rows:			
Export: Wrap Cell Content:			
	Query_ID	Duration	Query
▶	1	0.00024625	SHOW WARNINGS
	2	0.00307625	SELECT * FROM perf_innodb WHER...
	3	0.00718300	SELECT * FROM perf_myisam WHE...
	4	0.00154625	SELECT * FROM perf_memory WHE...
	5	0.00263600	SELECT * FROM perf_archive WHER...

-- Compare Multiple Engines

-- InnoDB Test

```
SET @start = NOW(6);
```

```
SELECT * FROM perf_innodb WHERE amount BETWEEN 10000 AND 50000;
```

```
SET @end = NOW(6);
```

```
317 • SELECT 'InnoDB' AS engine,
318 TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid		
Filter Rows:		
Export: Wrap Cell Content:		
	engine	ms
▶	InnoDB	1962.3990

-- MyISAM Test

```
SET @start = NOW(6);
```

```
SELECT * FROM perf_myisam WHERE amount BETWEEN 10000 AND 50000;
```

```
SET @end = NOW(6);
```

```
324 • SELECT 'MyISAM' AS engine,  
325     TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	engine	ms			
▶	MyISAM	2079.0440			

```
-- MEMORY Test
```

```
SET @start = NOW(6);
```

```
SELECT * FROM perf_memory WHERE amount BETWEEN 10000 AND 50000;
```

```
SET @end = NOW(6);
```

```
331 • SELECT 'MEMORY' AS engine,  
332     TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	engine	ms			
▶	MEMORY	1914.9860			

```
-- ARCHIVE Test
```

```
SET @start = NOW(6);
```

```
SELECT * FROM perf_archive WHERE amount BETWEEN 10000 AND 50000;
```

```
SET @end = NOW(6);
```

```
338 • SELECT 'ARCHIVE' AS engine,  
339     TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	engine	ms			
▶	ARCHIVE	4598.6500			

-- =====

-- 9. INDEX USAGE TEST

-- =====

344 • **EXPLAIN SELECT * FROM perf_innodb WHERE amount BETWEEN 10000 AND 20000;**

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	perf_innodb	NULL	range	idx_amount	idx_amount	6	NULL	1024	100.00	Using index condition

346 • **EXPLAIN SELECT * FROM perf_myisam WHERE amount BETWEEN 10000 AND 20000;**

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	perf_myisam	NULL	range	idx_amount	idx_amount	6	NULL	717	100.00	Using index condition

348 • **EXPLAIN SELECT * FROM perf_memory WHERE amount BETWEEN 10000 AND 20000;**

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	perf_memory	NULL	ALL	idx_amount	NULL	NULL	NULL	10000	11.11	Using where

350 • **EXPLAIN SELECT * FROM perf_archive WHERE amount BETWEEN 10000 AND 20000;**

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	perf_archive	NULL	ALL	NULL	NULL	NULL	NULL	10000	11.11	Using where

-- =====

-- 10. STORAGE SIZE CHECK

-- =====

```
356 • SELECT
357     table_name,
358     engine,
359     table_rows,
360     ROUND(data_length / 1024 / 1024, 2) AS data_mb,
361     ROUND(index_length / 1024 / 1024, 2) AS index_mb,
362     ROUND((data_length + index_length) / 1024 / 1024, 2) AS total_mb
363 FROM information_schema.tables
364 WHERE table_schema = 'storage_engine_lab'
365 ORDER BY total_mb DESC;
```

Result Grid				Filter Rows:		Export:		Wrap Cell Content:
	TABLE_NAME	ENGINE	TABLE_ROWS	data_mb	index_mb	total_mb		
▶	perf_memory	MEMORY	10000	4.16	0.49	4.65		
	perf_innodb	InnoDB	10196	0.52	0.27	0.78		
	perf_myisam	MyISAM	10000	0.31	0.22	0.52		
	perf_archive	ARCHIVE	10000	0.08	0.00	0.08		
	customers_innodb	InnoDB	0	0.02	0.00	0.02		
	customers_archive	ARCHIVE	0	0.00	0.00	0.00		
	customers_csv	CSV	2	0.00	0.00	0.00		
	customers_memory	MEMORY	0	0.00	0.00	0.00		
	customers_myisam	MyISAM	0	0.00	0.00	0.00		

-- Run once before testing (caching effect)

```
368 • SELECT COUNT(*) FROM perf_innodb;
```

Result Grid

Filter Rows:

Export:

Wrap

	COUNT(*)
▶	10000

First run - 0.015 sec / 0.000 sec

Second run - 0.000 sec / 0.000 sec

-- With index

```
371 • SELECT * FROM perf_innodb WHERE amount > 25000;
```

Result Grid				
Filter Rows:				
Edit:				
Export/Import:				
	id	customer_name	amount	created_at
▶	2861	Customer 2169	25010.05	2026-05-11 15:01:40
	5448	Customer 5542	25013.72	2026-05-11 15:01:40
	893	Customer 197	25014.33	2026-05-11 15:01:40
	8129	Customer 8821	25038.71	2026-05-11 15:01:40
	5633	Customer 5337	25058.12	2026-05-11 15:01:40
	5072	Customer 5978	25061.34	2026-05-11 15:01:40
	2113	Customer 2817	25061.59	2026-05-11 15:01:40
	0625	Customer 0225	25072.82	2026-05-11 15:01:40

-- Without index (force full scan)

```
374 • SELECT * FROM perf_innodb IGNORE INDEX (idx_amount)
375 WHERE amount > 25000;
376
```

Result Grid				
Filter Rows:				
Edit:				
Export/Import: Wrap				
	id	customer_name	amount	created_at
▶	1	Customer 909	85988.71	2026-05-11 15:01:40
	2	Customer 908	91465.86	2026-05-11 15:01:40
	3	Customer 907	99363.19	2026-05-11 15:01:40
	7	Customer 903	71774.55	2026-05-11 15:01:40
	8	Customer 902	50603.92	2026-05-11 15:01:40
	9	Customer 901	37695.98	2026-05-11 15:01:40
	10	Customer 900	26660.12	2026-05-11 15:01:40

SET @start = NOW(6);

SELECT * FROM perf_innodb WHERE amount > 25000;

SET @end = NOW(6);

```
380 • SELECT 'InnoDB' AS engine,
381 TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid	
Filter Rows:	
Export: Wrap Cell Content:	
engine	ms
▶ InnoDB	1577.2000

SET @start = NOW(6);

SELECT * FROM perf_innodb IGNORE INDEX (idx_amount)

WHERE amount > 25000;

```
SET @end = NOW(6);
```

```
387 • SELECT 'InnoDB' AS engine,  
388     TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
engine	ms		
InnoDB	2374.1570		

-- Index using faster than without index

-- Indexes are best when the query returns a small number of rows.

-- For large result sets, a full table scan can be faster.

```
SET @start = NOW(6);
```

```
SELECT * FROM perf_innodb
```

```
WHERE amount BETWEEN 25000 AND 25100;
```

```
SET @end = NOW(6);
```

```
398 • SELECT 'InnoDB With Index' AS test_name,  
399     TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
test_name	ms		
InnoDB With Index	3408.6140		

```
SET @start = NOW(6);
```

```
SELECT * FROM perf_innodb IGNORE INDEX (idx_amount)
```

```
WHERE amount BETWEEN 25000 AND 25100;
```

```
SET @end = NOW(6);
```

```
406 • SELECT 'InnoDB Without Index' AS test_name,  
407     TIMESTAMPDIFF(MICROSECOND, @start, @end)/1000 AS ms;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
test_name	ms		
InnoDB Without Index	3347.7690		